

Software Ingeniaritza II

PATTERNS



Iván González

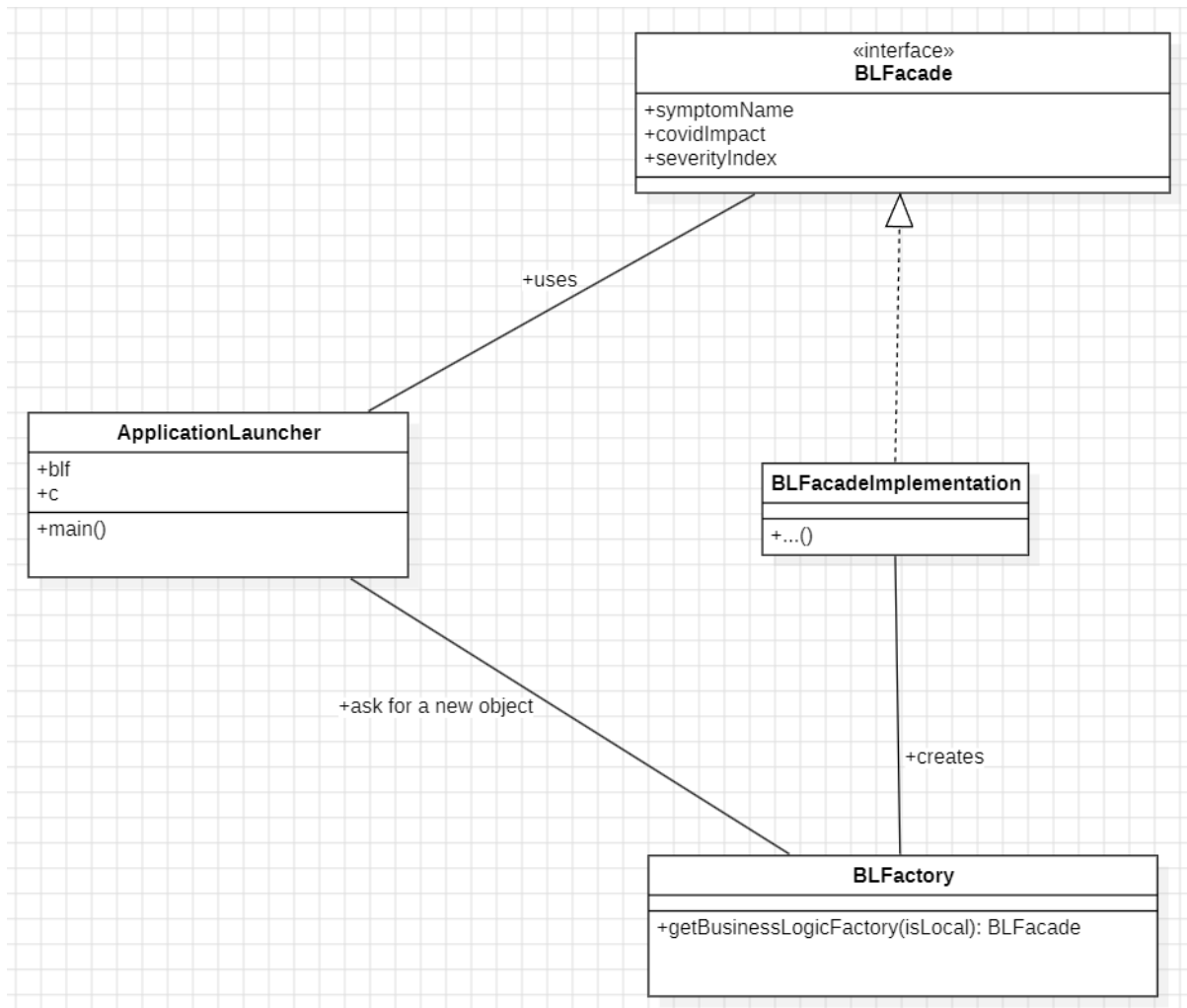
Beñat Arruti

Mikel Berlanga

Eneko Santos

Factory method

UML:



Implementazioa:

```
package businessLogic;
import java.net.URL;

public class BLFactory {
    ConfigXML c = ConfigXML.getInstance();

    public BLFactory() {}

    public BLFacade getBusinessLogicFactory(boolean isLocal) {
        try {
            if (isLocal == true) {
                DataAccess da = new DataAccess();
                return new BLFacadeImplementation(da);
            } else {
                String serviceName = "http://" + c.getBusinessLogicNode() + ":" + c.getBusinessLogicPort() + "/ws/"
                    + c.getBusinessLogicName() + "?wsdl";

                URL url = new URL(serviceName);

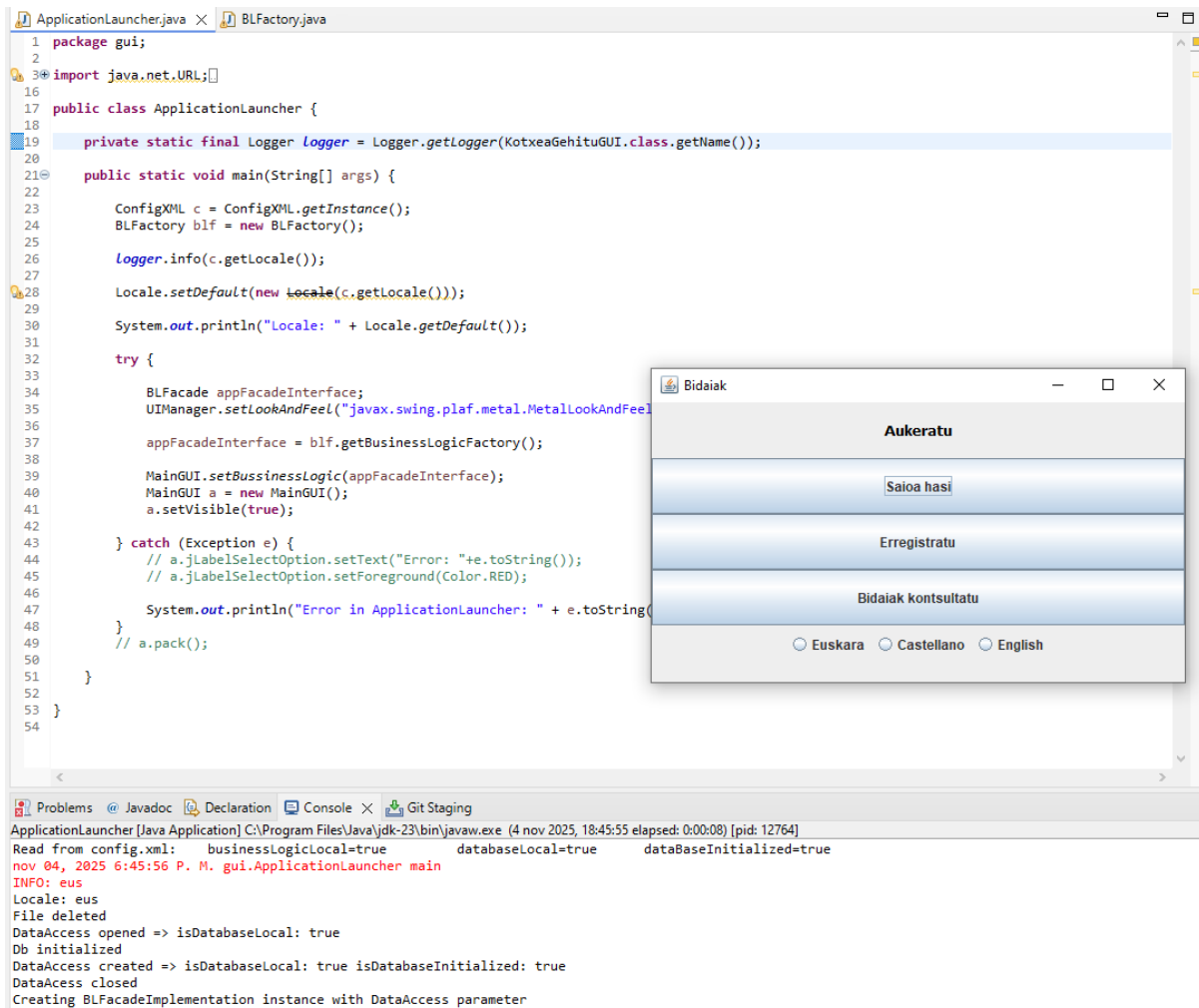
                // 1st argument refers to wsdl document above
                // 2nd argument is service name, refer to wsdl document above
                QName qname = new QName("http://businessLogic/", "BLFacadeImplementationService");

                Service service = Service.create(url, qname);

                return service.getPort(BLFacade.class);
            }
        } catch (Exception e) {
            return null;
        }
    }
}
```

BLFacade-a sortzeko klase bat sortu dut. Application launcher-ak klase honi BLFacadea sortzeko eskatuko dio, eta parametro baten bidez adieraziko da ea lokala izango den ala ez.

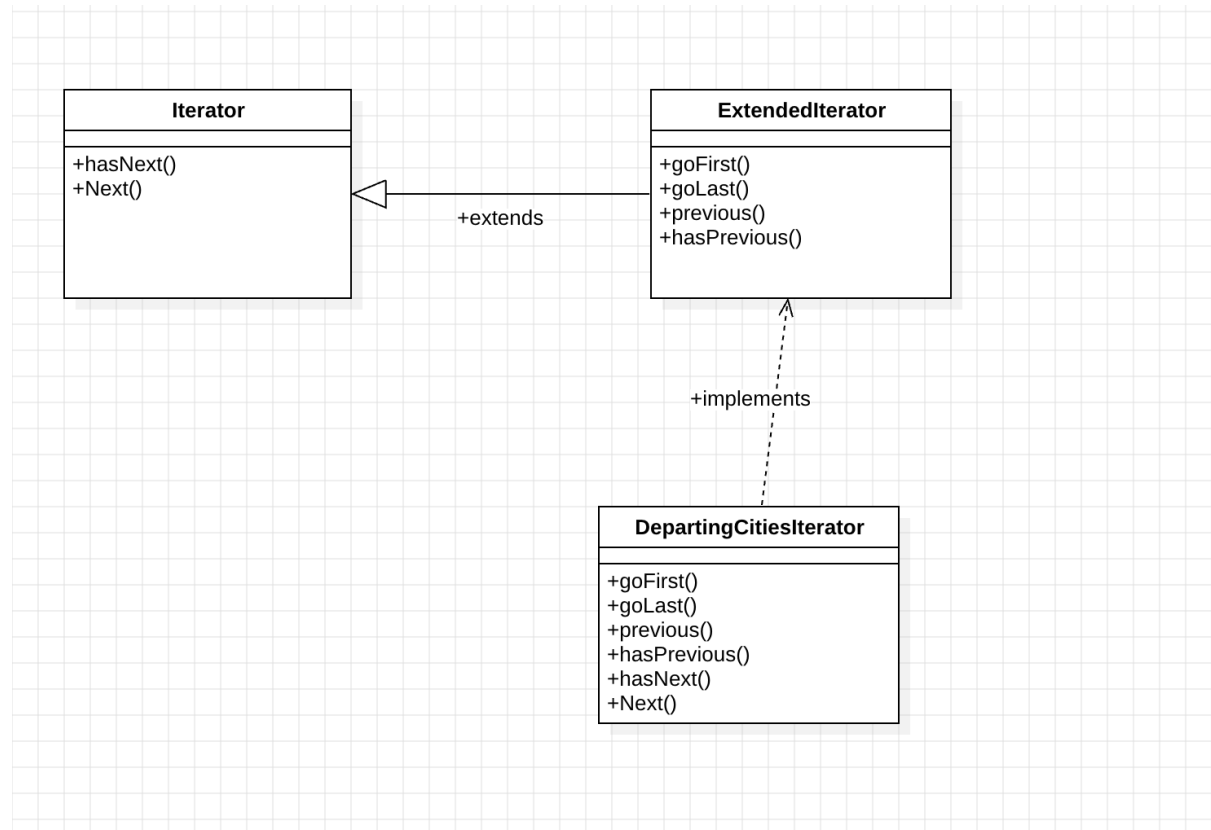
Proba Exekuzioa:



Ikusi dezakegu nola programa funtzionatzen duen BLFactory-rekin.

Iterator

UML:



Implementazioa:

```
public class DepartingCitiesIterator implements ExtendedIterator<String> {
```

```
    private java.util.List<String> cities;
```

```
    private int position = -1;
```

```
    public DepartingCitiesIterator(java.util.List<String> cities) {
```

```
        this.cities = cities;
```

```
    }
```

```
    @Override
```

```
    public boolean hasNext() {
```

```
return position < cities.size();  
}
```

```
@Override  
public String next() {  
    String city = cities.get(position);  
    position++;  
    return city;  
}
```

```
@Override  
public String previous() {  
    String city = cities.get(position);  
    position--;  
    return city;  
  
}
```

```
@Override  
public boolean hasPrevious() {  
    return position >= 0;  
}
```

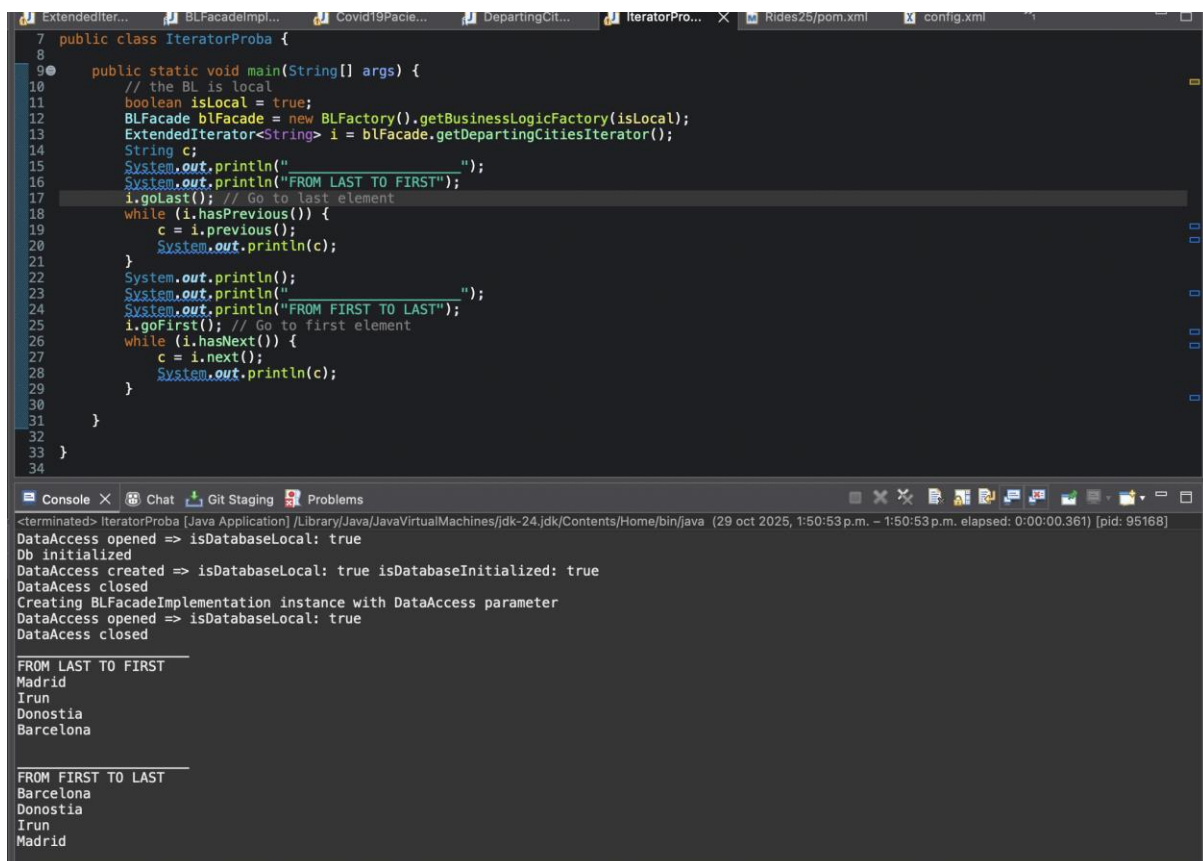
```
@Override  
public void goFirst() {  
    position = 0;  
}
```

@Override

```
public void goLast() {  
  
    position = cities.size()-1;  
  
}  
  
}
```

City klasera egokitutako iterator klasea sortu dut.

Proba Exekuzioa:



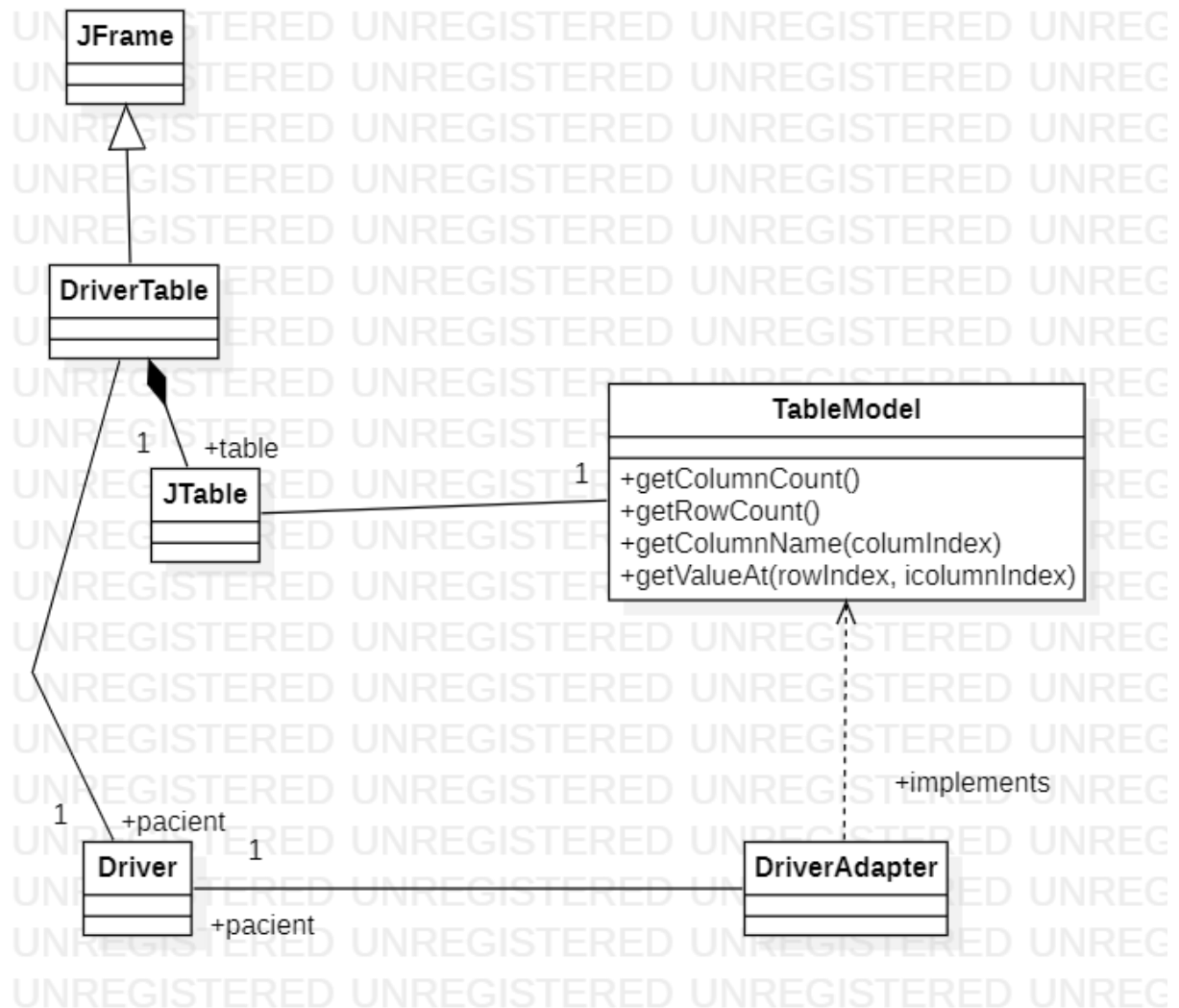
```
7 public class IteratorProba {  
8  
9     public static void main(String[] args) {  
10         // the BL is local  
11         boolean isLocal = true;  
12         BLFacade blFacade = new BLFactory().getBusinessLogicFactory(isLocal);  
13         ExtendedIterator<String> i = blFacade.getDepartingCitiesIterator();  
14         String c;  
15         System.out.println(" ");  
16         System.out.println("FROM LAST TO FIRST");  
17         i.goLast(); // Go to last element  
18         while (i.hasPrevious()) {  
19             c = i.previous();  
20             System.out.println(c);  
21         }  
22         System.out.println();  
23         System.out.println(" ");  
24         System.out.println("FROM FIRST TO LAST");  
25         i.goFirst(); // Go to first element  
26         while (i.hasNext()) {  
27             c = i.next();  
28             System.out.println(c);  
29         }  
30     }  
31 }  
32  
33 }  
34
```

Console Output:

```
<terminated> IteratorProba [Java Application] /Library/Java/JavaVirtualMachines/jdk-24.jdk/Contents/Home/bin/java (29 oct 2025, 1:50:53 p.m. - 1:50:53 p.m. elapsed: 0:00:00.361) [pid: 95168]  
DataAccess opened => isDatabaseLocal: true  
Db initialized  
DataAccess created => isDatabaseLocal: true isDatabaseInitialized: true  
DataAccess closed  
Creating BLFacadeImplementation instance with DataAccess parameter  
DataAccess opened => isDatabaseLocal: true  
DataAccess closed  
  
FROM LAST TO FIRST  
Madrid  
Irun  
Donostia  
Barcelona  
  
FROM FIRST TO LAST  
Barcelona  
Donostia  
Irun  
Madrid
```

Adapter

UML:



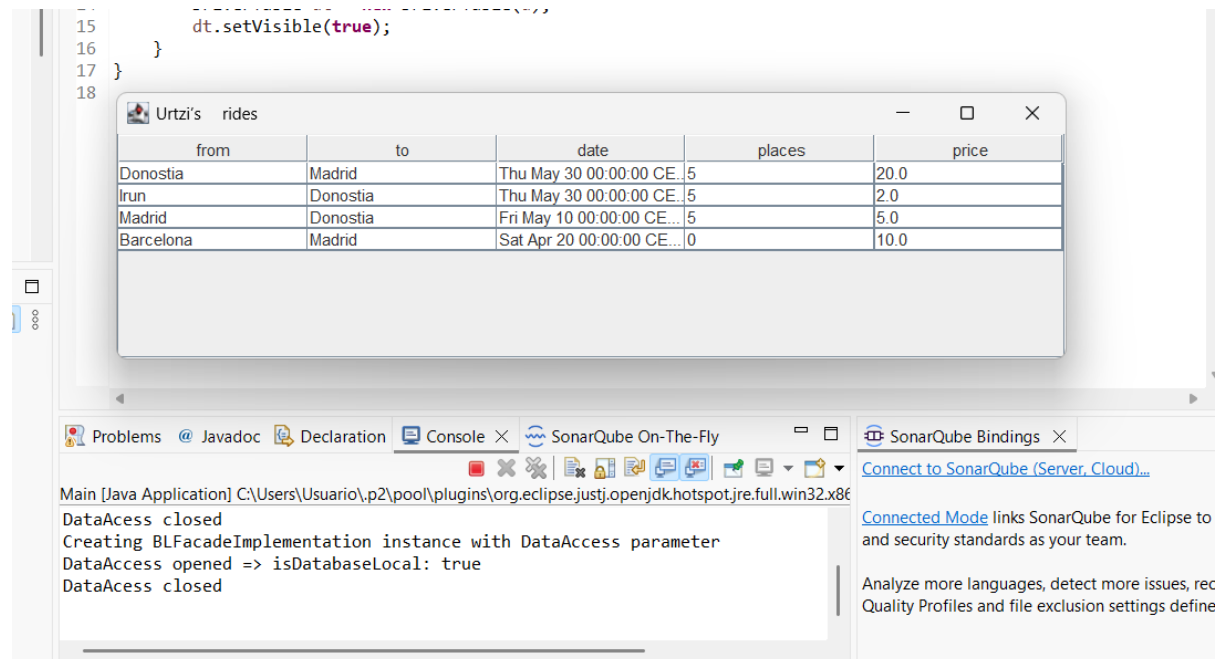
Implementazioa:

```
public class DriverAdapter extends AbstractTableModel {
    private static final long serialVersionUID = 1L;
    protected Driver driver;
    protected String[] columnNames = new String[] {"from", "to", "date", "places", "price"};
    public DriverAdapter(Driver driver) {
        this.driver = driver;
    }
    public int getColumnCount() {
        return columnNames.length;
    }

    public int getRowCount() {
        List<Ride> rides = driver.getCreatedRides();
        return rides == null ? 0 : rides.size();
    }
    public String getColumnName(int columnIndex) {
        return columnNames[columnIndex];
    }
    public Object getValueAt(int rowIndex, int columnIndex) {
        List<Ride> rides = driver.getCreatedRides();
        if (rides == null || rowIndex < 0 || rowIndex >= rides.size()) return null;
        Ride r = rides.get(rowIndex);
        switch (columnIndex) {
            case 0:
                return r.getFrom();
            case 1:
                return r.getTo();
            case 2:
                return r.getDate();
            case 3:
                return r.getnPlaces();
            case 4:
                return r.getPrice();
            default:
                return null;
        }
    }
}
```

DriverAdapter klasea sortu eskatzen duen taula eguneratzeko. Klase hobetan lau metoso daude taula hori eraikitzeko.

Proba Exekuzioa:



The screenshot shows the Eclipse IDE interface. At the top, a Java code editor displays a snippet of code with line numbers 15 to 18. Line 15 contains `dt.setVisible(true);`. Below the code editor, a window titled "Urtzi's rides" is open, displaying a table with the following data:

from	to	date	places	price
Donostia	Madrid	Thu May 30 00:00:00 CE...	5	20.0
Irun	Donostia	Thu May 30 00:00:00 CE...	5	2.0
Madrid	Donostia	Fri May 10 00:00:00 CE...	5	5.0
Barcelona	Madrid	Sat Apr 20 00:00:00 CE...	0	10.0

Below the table window, the Eclipse IDE's bottom section is visible. The "Console" tab is active, showing the following output:

```
Main [Java Application] C:\Users\Usuario\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jre\bin\java.exe: -Djava.library.path=C:\Users\Usuario\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64.jre\bin
DataAccess closed
Creating BLFacadeImplementation instance with DataAccess parameter
DataAccess opened => isDatabaseLocal: true
DataAccess closed
```

To the right of the console, the "SonarQube Bindings" tab is visible, showing a "Connect to SonarQube (Server, Cloud)..." button and a "Connected Mode" link. A sidebar on the far right contains text about analyzing more languages and detecting more issues.